Fakultas Informatika
School of Computing
Telkom University

# CAK2HAB3 - Dasar Kecerdasan Artifisial

## Searching: Introduction

Lecturer Team

# Outline

- Why Searching?
- Space Representation
- Search Space

# Why Searching?

# Natural Intelligence

- Knowledge comes from learning.
- Learning comes from searching for the right information

- Search is a universal problem-solving mechanism in AI.

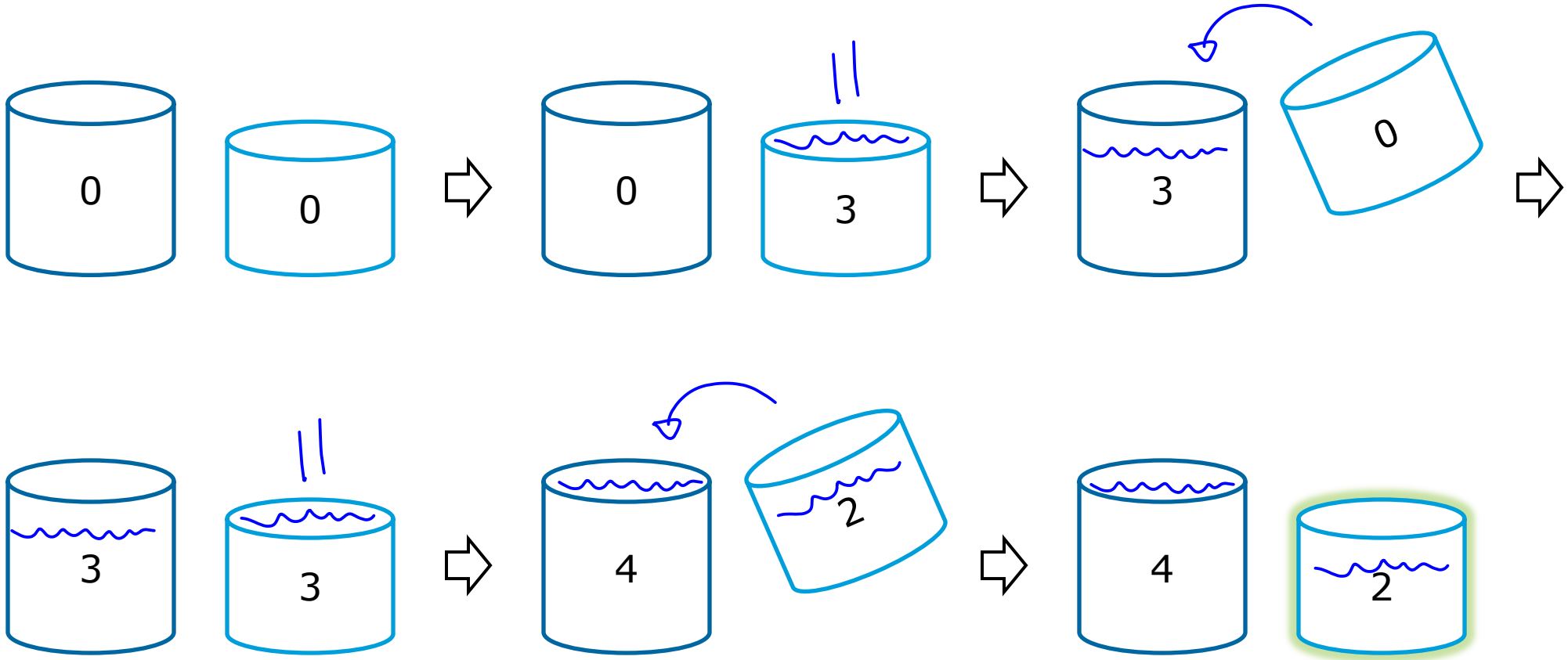
# The Water Jug Problem

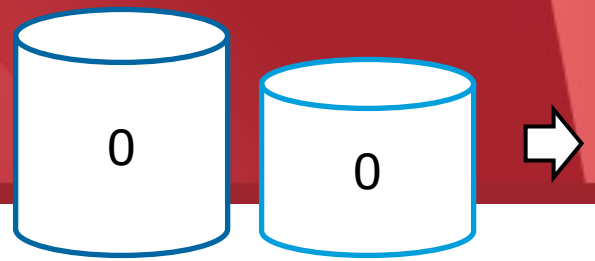

There's

- A **4-gallon** and a **3-gallon** water container
- An infinite water fountain

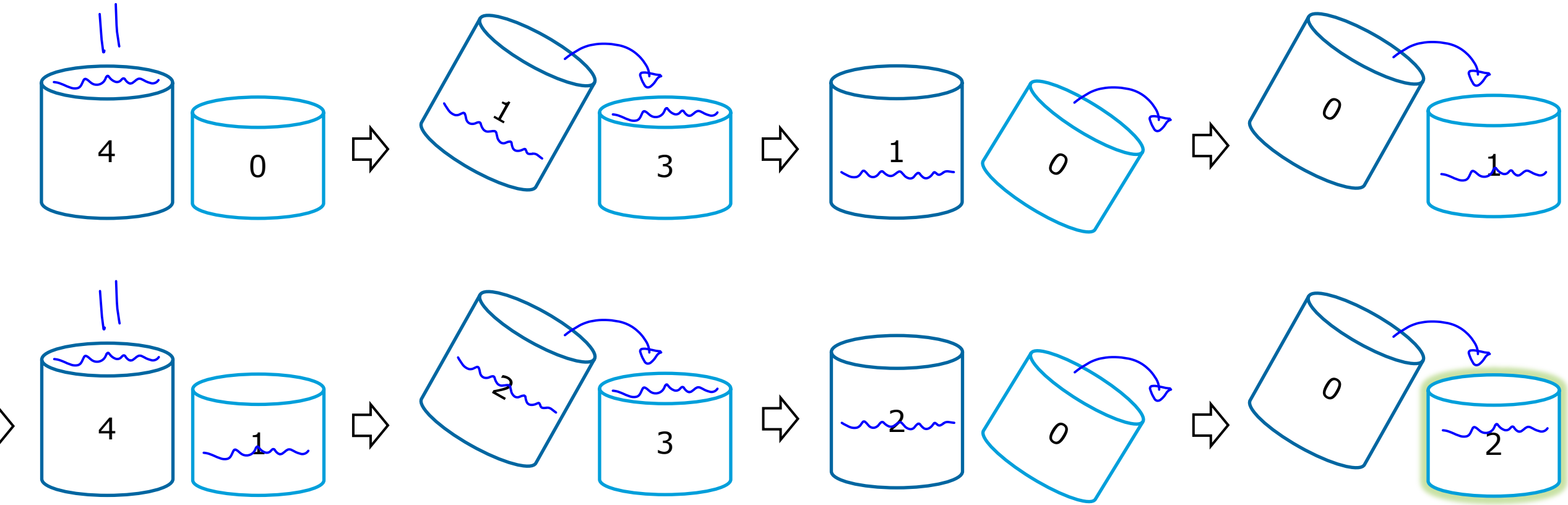
- Fill **exactly 2 gallons** of water inside the **3-gallon** container

# 1st Solution






# 2nd Solution



**You can do it,**
**the goal is to tell Computer to do the same**

# State Spaces

- Many AI problems can be readily modeled as state spaces
- And solving these problems can "simply" be reduced to exploring the state space,
  and identifying the correct answer.
- That is search

# State Spaces Representation

- Converting the given problem space/situation into another form of space using a certain operations so that can help to formulate the solution
- Searching
  - Representing (modelling) a problem into state space
- Engineering skills are required

# State Spaces – jug problem

- 2 types of container
  - X and Y → (X,Y)
- Possible states
  - X = {0, 1, 2, 3, 4}
  - Y = {0, 1, 2, 3}
- Initial state = (0, 0)
- Goal state = (n, 2)
  - n = [0, 4]

# Operation set

- Operators
  - Steps to turn a state into another state
- Operation set must be complete
  - An incomplete set may not achieve any solution
  - How to identify the completeness?

# Operation set – jug problem

| # | Operation | Condition (if) | Final State |
|---|---|---|---|
| 1 | Fully filled the 4-gallon container | X < 4 | (4, Y) |
| 2 | Fully filled the 3-gallon container | Y < 3 | (X, 3) |
| 3 | Empty the 4-gallon container | X > 0 | (0, Y) |
| 4 | Empty the 3-gallon container | Y > 0 | (X, 0) |
| | | | |

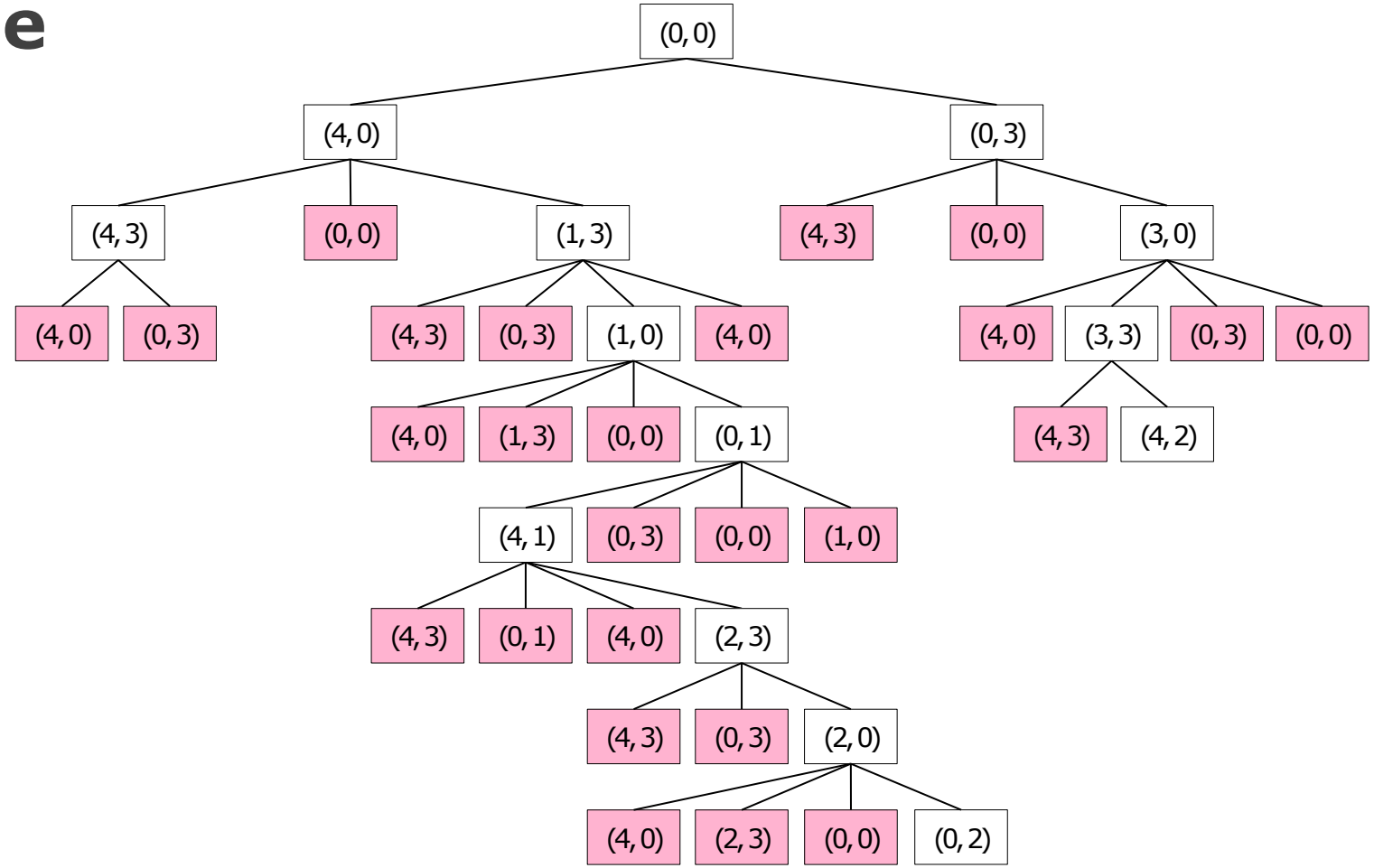
# Operation set – jug problem

| # | Operation | Condition (if) | Final State |
|---|---|---|---|
| 5 | Move some water from the 3-gallon container into the 4-gallon container until it's fully filled | (X+Y ≥ 4) && (Y > 0) | (4, Y-(4-X)) |
| 6 | Move some water from the 4-gallon container into the 3-gallon container until it's fully filled | (X+Y ≥ 3) && (X > 0) | (X-(3-Y), 3) |
| 7 | Move all water from the 3-gallon container into the 4-gallon container | (X+Y ≤ 4) && (Y > 0) | ( X+Y, 0 ) |
| 8 | Move all water from the 4-gallon container into the 3-gallon container | (X+Y ≤ 3) && (X > 0) | ( 0, X+Y ) |

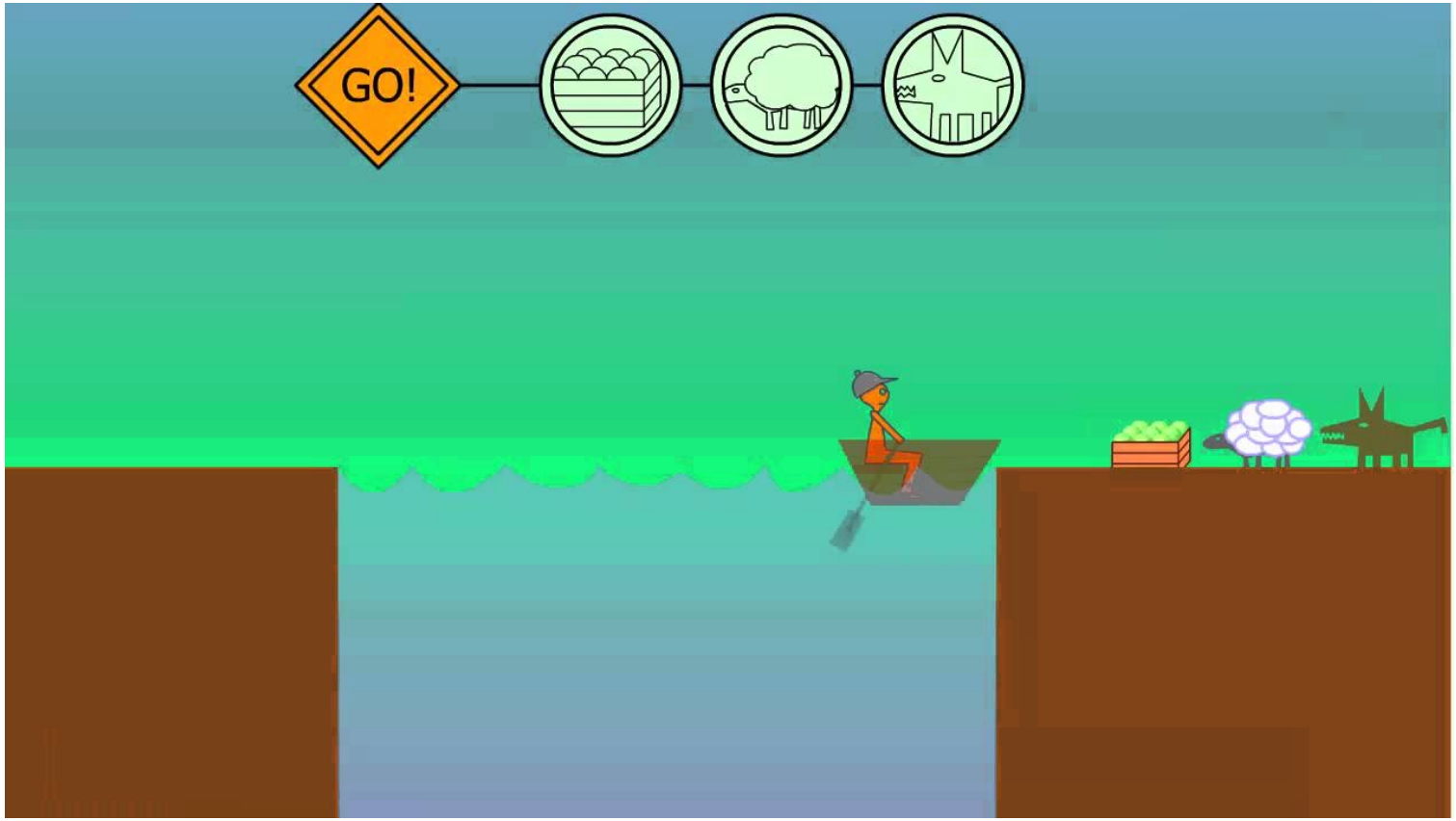
# Solution – jug problem

| Operator used | 4-gallon container | 3-gallon container |
|---|---|---|
| start | 0 | 0 |
| 2 | 0 | 3 |
| 7 | 3 | 0 |
| 2 | 3 | 3 |
| 5 | 4 | 2 |

# Solution Space



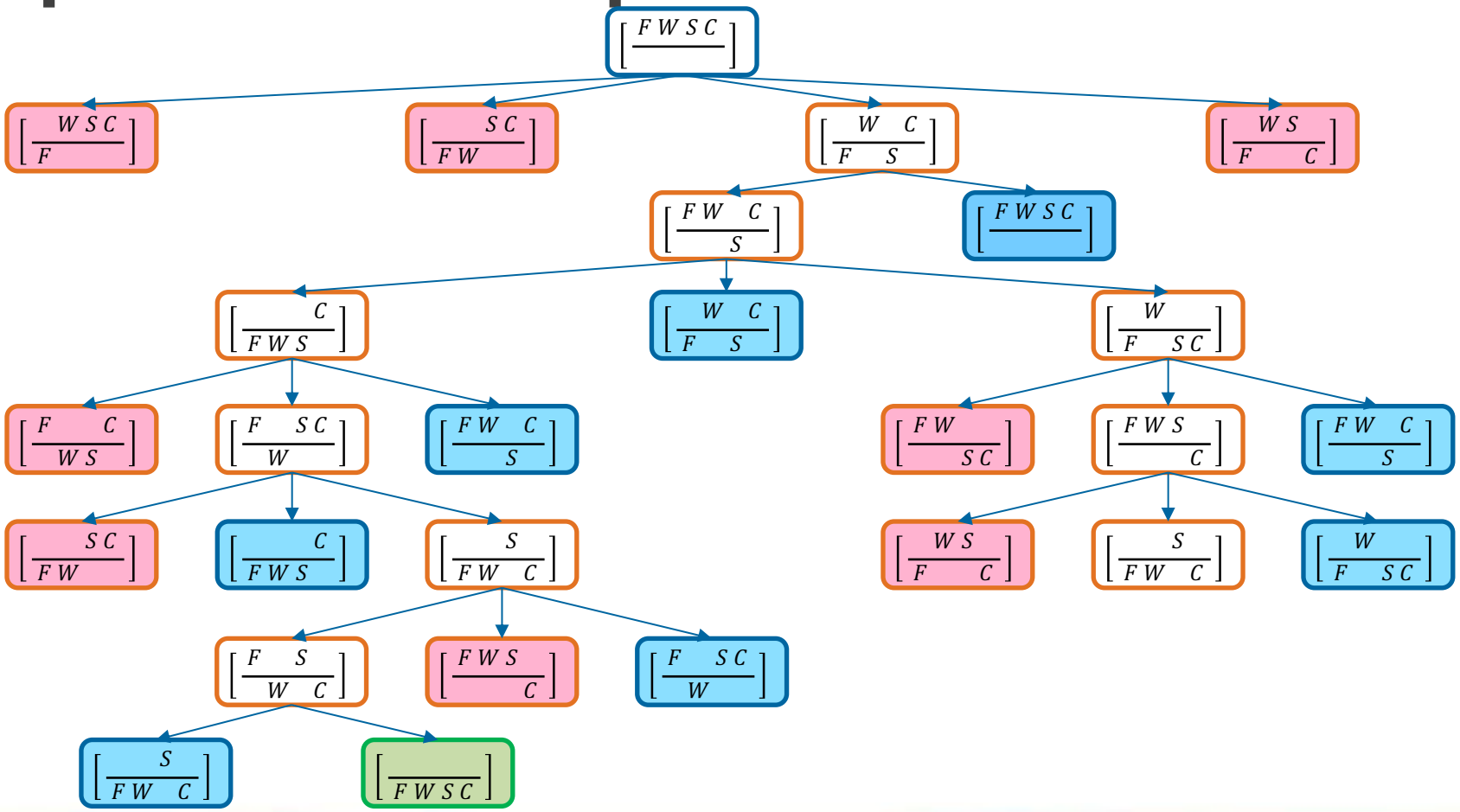
# Farmer Problems



# State Spaces – farmer problem

- 4 objects
  - Farmer F
  - Wolf W
  - Sheep S
  - Cabbage C
- Possible states
  - The Location
- Constraints
- Initial state
  - $\left( \frac{F\ W\ S\ C}{} \right)$
- Goal state
  - $\left( \frac{}{F\ W\ S\ C} \right)$

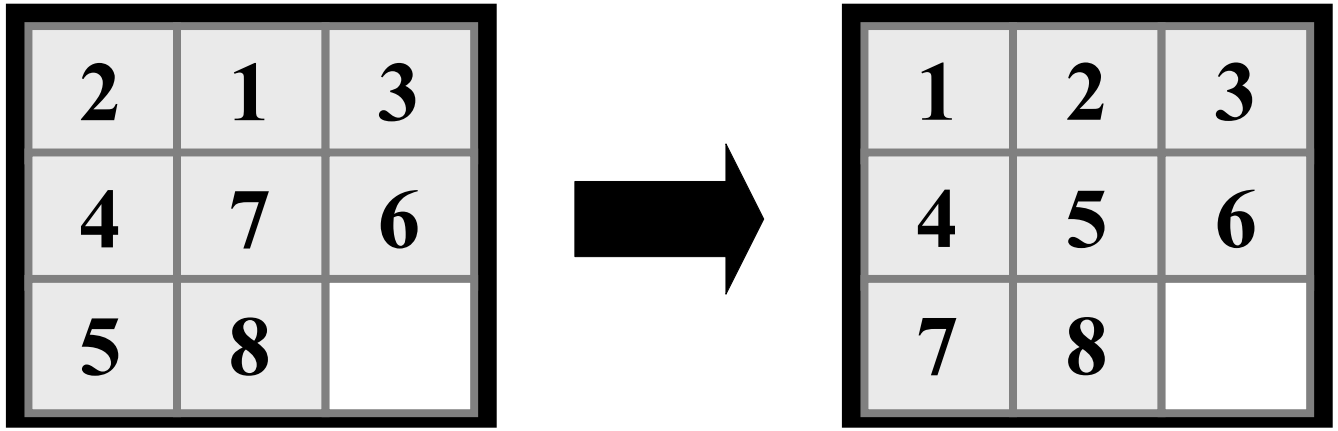
# Solution Spaces – farmer problem



# Operation set – farmer problem

| # | Operation |
|---|---|
| 1 | Move Farmer down |
| 2 | Move Farmer up |
| 3 | Move Farmer with Wolf down |
| 4 | Move Farmer with Wolf up |
| 5 | Move Farmer with Sheep down |
| 6 | Move Farmer with Sheep up |
| 7 | Move Farmer with Cabbage down |
| 8 | Move Farmer with Cabbage up |
| 9 | … ? |

# N-Puzzle Problem



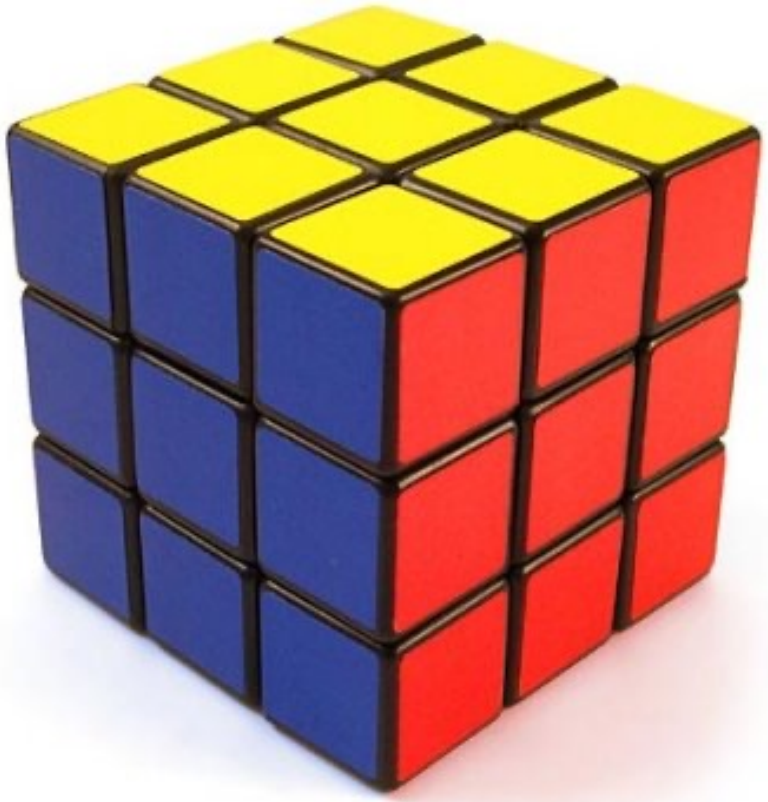

*Initial State* *Goal State*

# Operation set – 8-puzzle problem

| # | Operation 1 |
|---|---|
| 1 | Move 1 left |
| 2 | Move 1 right |
| 3 | Move 1 up |
| 4 | Move 1 down |
| 5 | Move 2 left |
| 6 | Move 2 right |
| 7 | Move 2 up |
| 8 | Move 2 down |
| … | |
| … | |
| … | |
| 32 | Move 8 down |

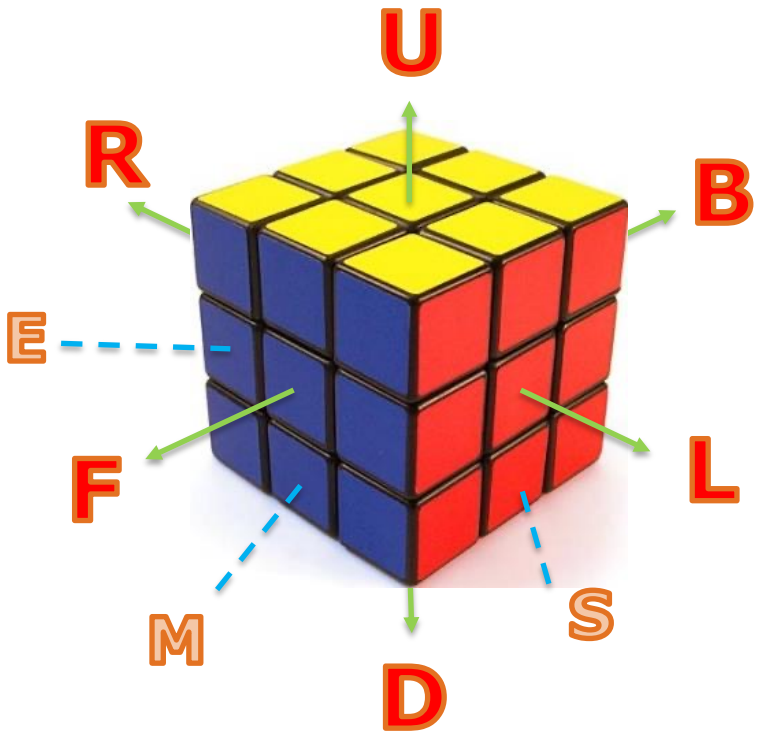
| # | Operation 2 |
|---|---|
| 1 | Move blank left |
| 2 | Move blank right |
| 3 | Move blank up |
| 4 | Move blank down |

# Rubik's Cube Problem

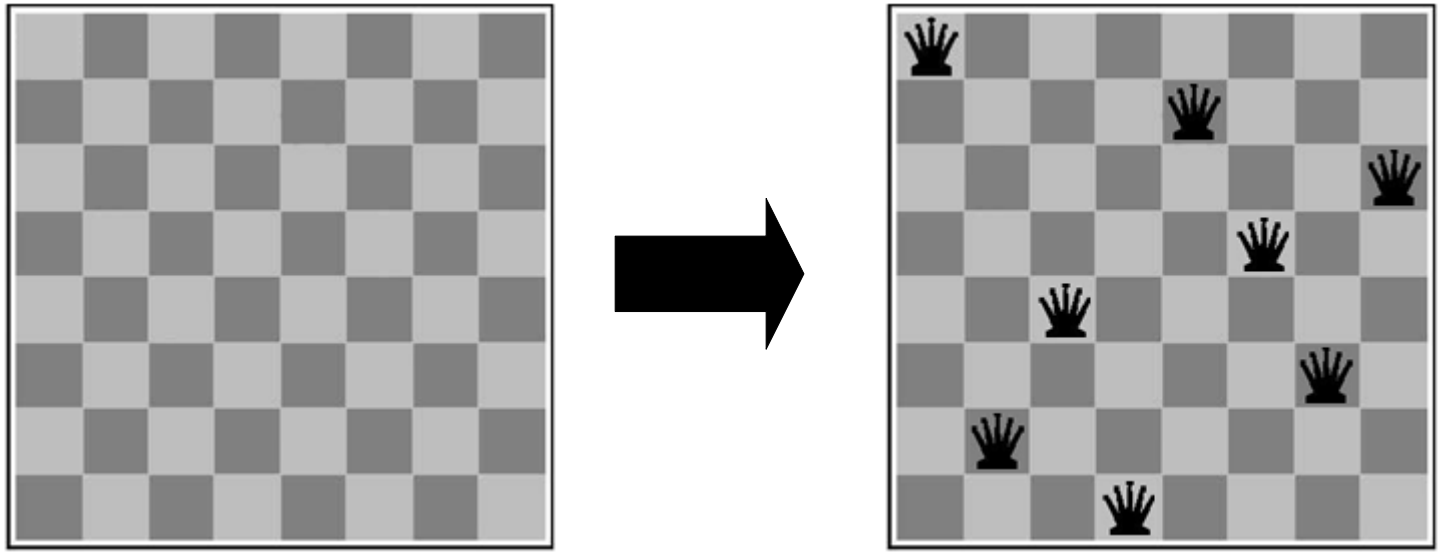


# Operation set – 3x3 Rubik's Cube

| # | Operation |
|---|---|
| 1 | 1L |
| 2 | 2L |
| 3 | 3L |
| 4 | 1M |
| 5 | 2M |
| 6 | 3M |
| 7 | 1R |
| 8 | 2R |
| … | |
| … | |
| … | |
| 27 | 3D |



# N-Queen Problem



*Initial State* *Goal State*

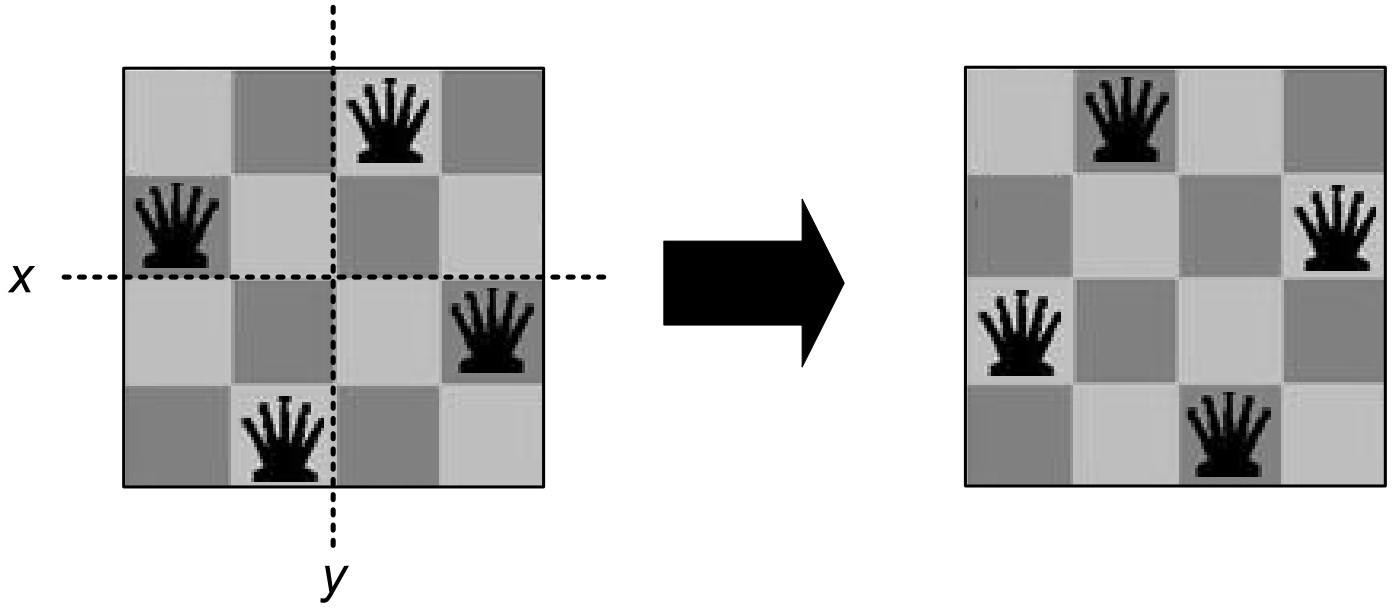
# N-Queen Problem

- problem of placing n non-attacking queens on an $n \times n$ chessboard
  - with the exception of n=2 and n=3
- a solution requires that no two queens share the same row, column, or diagonal

# N-Queen Problem

- computationally expensive
- For 8-queens problem
  - ${}_{64}C_8 = (64 \times 63 \times \ldots \times 58 \times 57)/8!$ possible arrangements
  - Or **4,426,165,368** possible arrangements
  - But only **92 possible solutions**

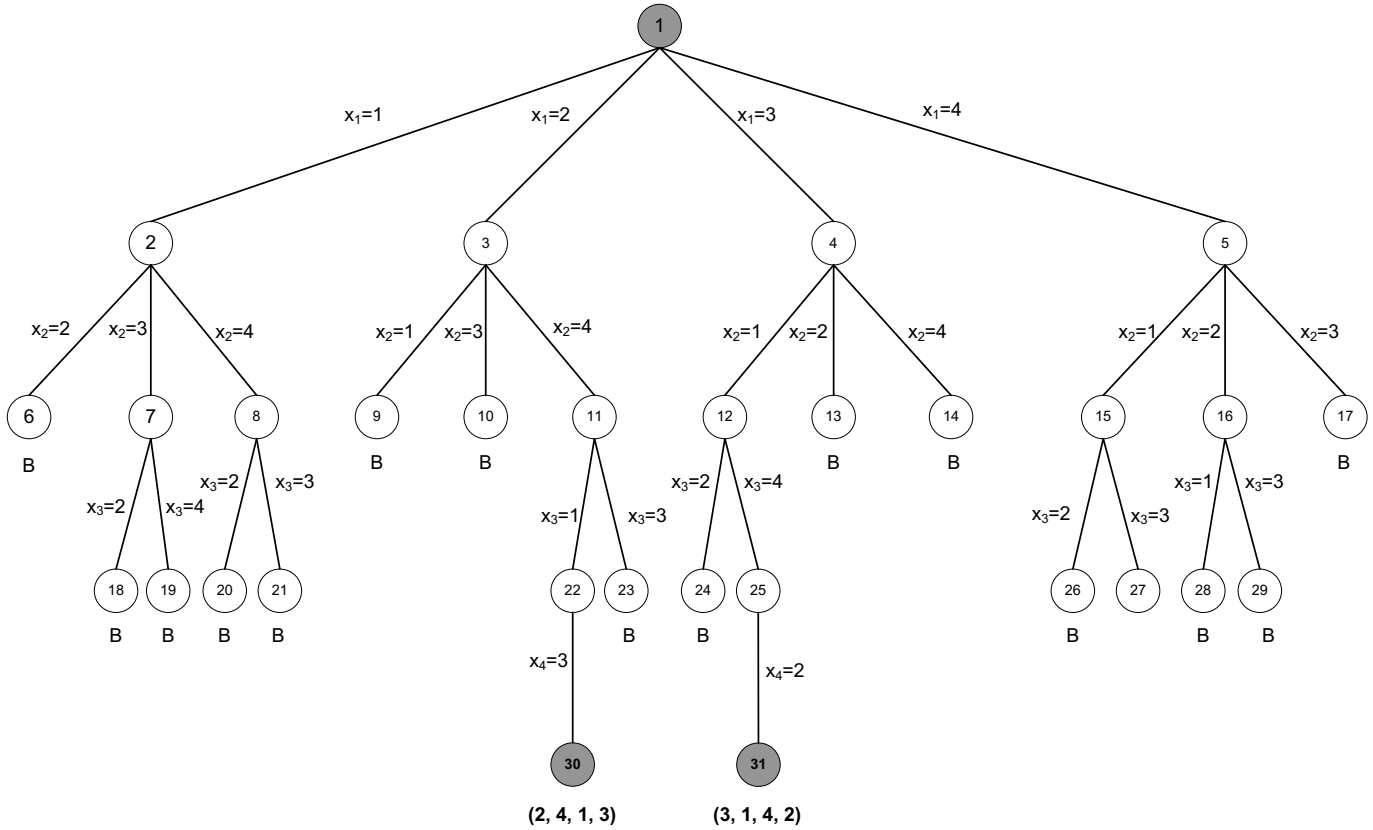
# 4-Queens Problem




Solusi 1

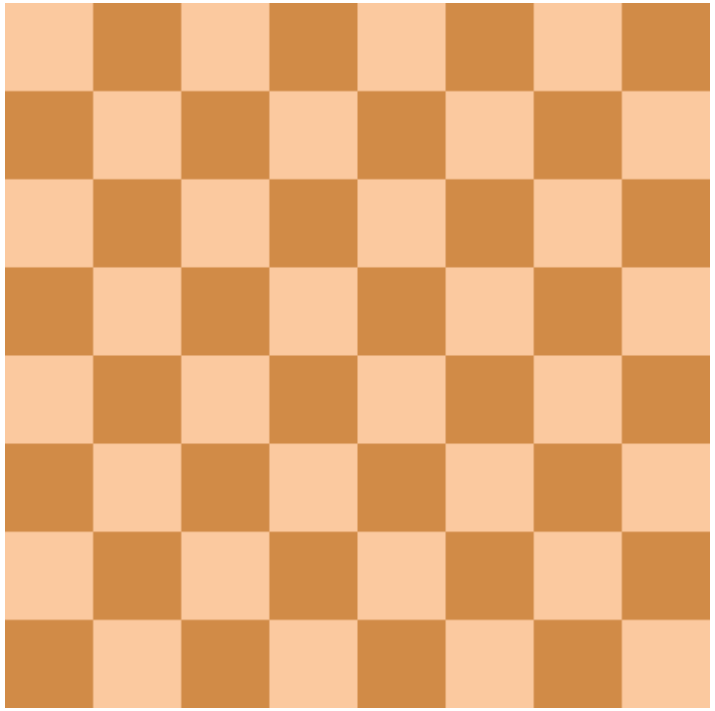
Solusi 2

# 4-Queens Problem Solution Space



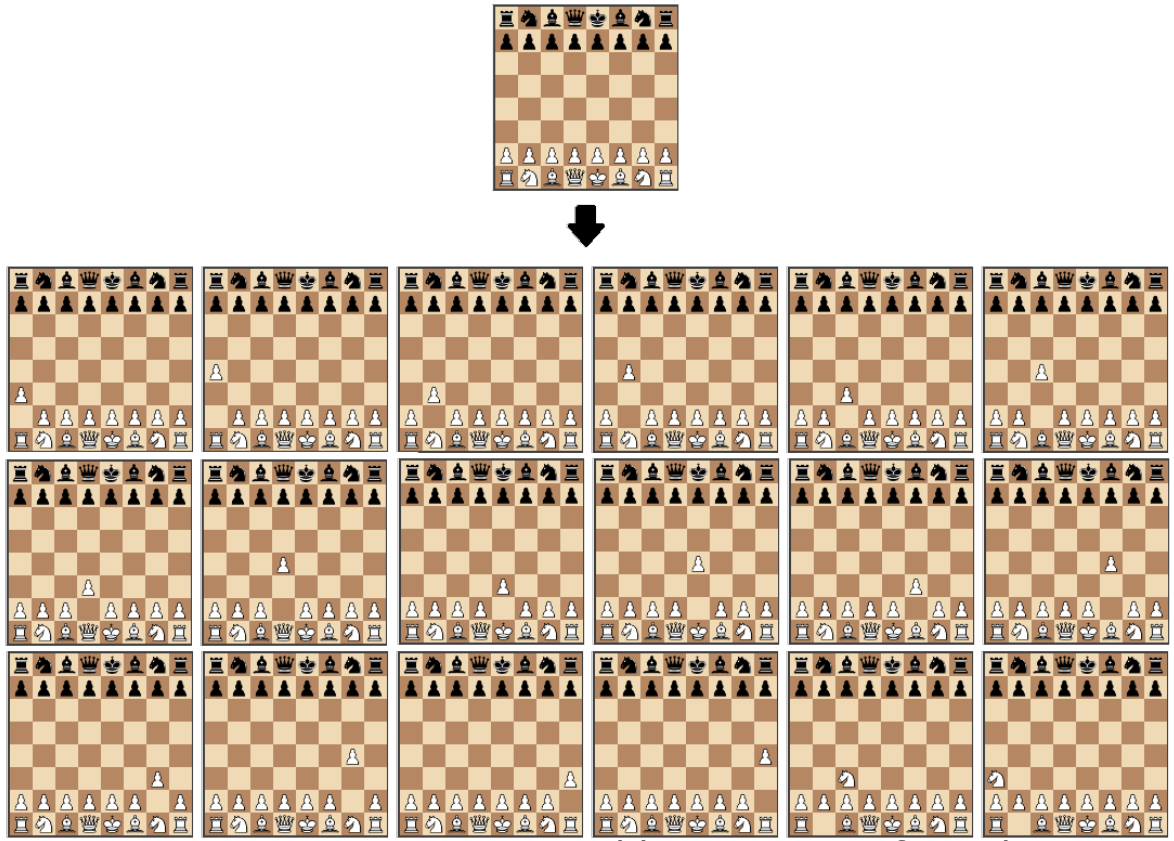
| N | # of Possible Solutions | # of Unique Solutions |
|---|---|---|
| 1 | 1 | 1 |
| 2 | 0 | 0 |
| 3 | 0 | 0 |
| 4 | 2 | 1 |
| 5 | 10 | 2 |
| 6 | 4 | 1 |
| 7 | 40 | 6 |
| 8 | 92 | 12 |
| 9 | 352 | 46 |
| 10 | 724 | 92 |
| 11 | 2.680 | 341 |
| 12 | 14.200 | 1.787 |
| 13 | 73.712 | 9.233 |
| 14 | 365.596 | 45.752 |
| 15 | 2.279.184 | 285.053 |
| ... | ... | ... |
| 23 | 24.233.937.684.440 | 3.029.242.658.210 |

# Backtrack Search solution example



# Chess Game

# Chess Game



20 possible position in first ply move

# Chess Game Operation Set

- Enormous operations
  - 400 possible chess positions after two ply moves
    (first ply move for White followed by first ply move for Black).
  - 5,362 possible positions after white's second ply move
  - 8,902 total positions after two ply moves each

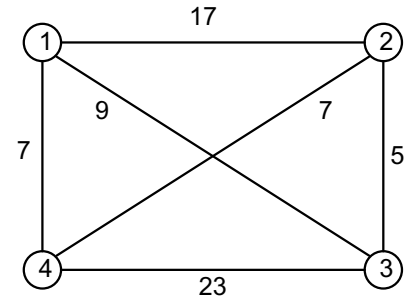
- **±10,921,506** total possible positions only after
  7 ply moves

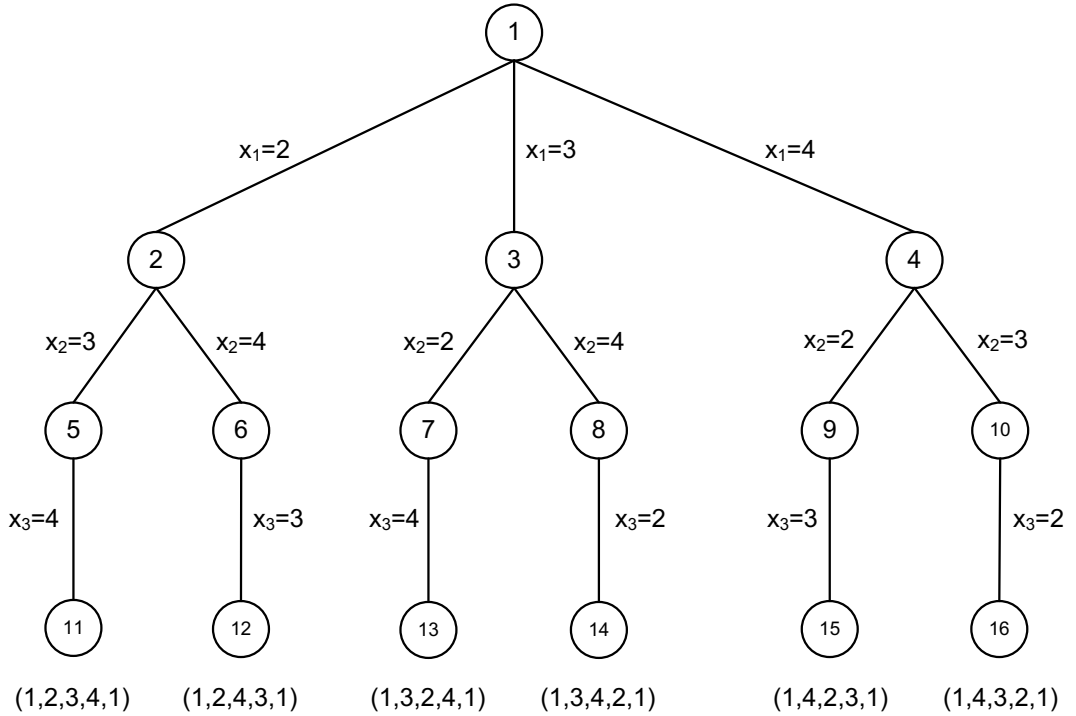
# Travelling Salesman Problem

# Travelling Salesman Problem

- "Given a list of cities and the distances between each pair of cities, what is the shortest possible route that visits each city exactly once and returns to the origin city?"
  - Search the order of all the locations to visit
  - Start from a city and return to the city
  - Minimize total cost.
  - Every city must be visited once.

# TSP Solution Space



# Search-based Application

- Modelling a problem into a state space
- Production System:
  - A set of rules/operations
  - One or more knowledge or databases containing any information for a particular purpose.
  - Control strategy (searching method)
    - Specifies the order in which the rules will be compared to the database;
    - Determine how to troubleshoot if some rules can be done at the same time.

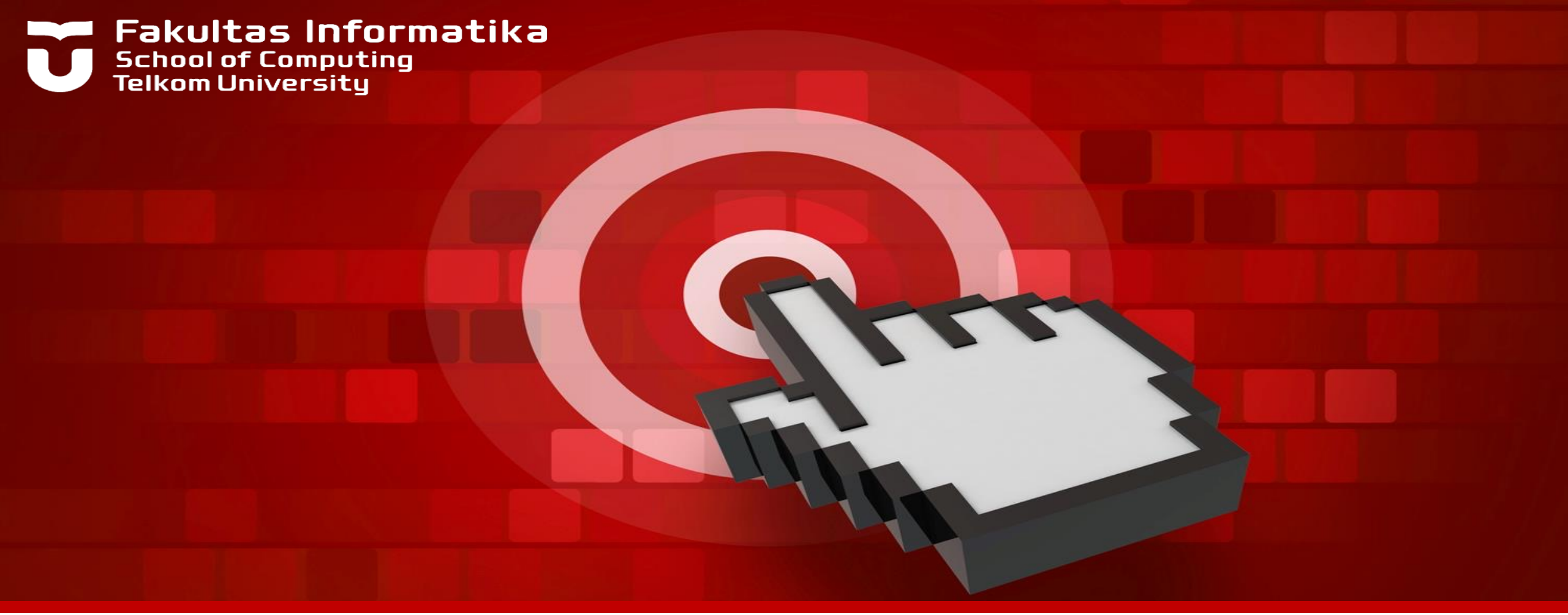
# Searching Algorithms

- Blind Search (Uninformed Search)
  - No prior information
  - High complexity
- Heuristic Search (Informed Search)
  - Provided prior information
  - Relatively low complexity

# Performance Measurements

- Completeness
  - Does the method guarantee the discovery of a solution if exists?
- Optimality
  - Does the method guarantee finding the best solution if there are many different solutions?
- Time complexity
  - How long will it take?
- Space complexity
  - How much memory is needed?

# Question?



# THANK YOU